

Algoritmi Salso e Greedy Search

Algoritmi a confronto per l'ottimizzazione bayesiana del clustering dei dati attraverso la teoria delle decisioni

Tommaso Menghini Tommaso Pozzi

Esame Statistica Non Parametrica, 2024

Indice-I

1 Introduzione

- DPMM per problemi di clustering
- Sintetizzare la distribuzione a posteriori

2 Funzione di Perdita

- Funzione di perdita Binder
- Funzione di perdita VI

Indice-II

3 Algoritmi

- Teoria del Reticolo
- Algoritmo Greedy Search
- Credible Balls
- Algoritmo Salso

4 Studio di Simulazione

5 Esempio su dati reali

DPMM per problemi di clustering

- I metodi di clustering non parametrici offrono un approccio flessibile per individuare strutture nei dati, senza dover specificare a priori il numero di gruppi.
- Tra questi, i modelli di mistura assumono un numero di componenti teoricamente infinito, cioè il numero di componenti che aumenta all'aumentare delle osservazioni.
- L'analisi dei dati tramite DPMM consente di esplorare la distribuzione a posteriori delle partizioni latenti.



Individuare uno stimatore puntuale e la sua incertezza

- Problema: Individuare una partizione dei dati che sia rappresentativa della distribuzione a posteriori.
- Si propone una soluzione che adotta un approccio basato sull'uso di funzioni di perdita. In particolare la perdita *Binder* e *VI*.
- Un vantaggio del framework bayesiano è la capacità di quantificare l'incertezza delle stime prodotte. Si approfondiscono due approcci per l'analisi dell'incertezza: la *posterior similarity matrix* e le *credible balls*.

Binder

- La funzione di perdita Binder:

$$B(\mathbf{c}, \hat{\mathbf{c}}) = \sum_{n < n'} (a \cdot \mathbf{1}\{c_n = c_{n'}\} \mathbf{1}\{\hat{c}_n \neq \hat{c}_{n'}\} + b \cdot \mathbf{1}\{c_n \neq c_{n'}\} \mathbf{1}\{\hat{c}_n = \hat{c}_{n'}\})$$

- Per ogni possibile coppia di osservazioni si penalizzano due tipologie di errori con pesi diversi a e b .
- La versione invariante rispetto ad N , assume $a = b$:

$$B(\mathbf{c}, \hat{\mathbf{c}}) = \sum_{i=1}^{k_N} \left(\frac{n_{i+}}{N}\right)^2 + \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{+j}}{N}\right)^2 - 2 \sum_{i=1}^{k_N} \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{ij}}{N}\right)^2$$

- Si dimostra che la partizione $\mathbf{c}^*$ che minimizza l'espressione sopra è la stessa che minimizza:

$$LS(\mathbf{c}, \hat{\mathbf{c}}) = \sum_{n < n'} (\mathbf{1}(c_n = c_{n'}) - \pi_{nn'})^2$$

Binder Generalizzata

- Problema: Il partizionamento ottimale basato sulla Binder Loss sovrastima il numero di cluster.
- La funzione di perdita Binder è stata introdotta con a e b diversi. Viene allora fornita una versione invariante rispetto a N che ripristina la flessibilità di scelta per i pesi.
- Si dimostra che:
 - Non è computazionalmente più complessa;
 - Se si pone $a > b$ si risolve il problema di sovrastima dei cluster;
- L'espressione per la funzione di perdita Binder Generalizzata è

$$B(\mathbf{c}, \hat{\mathbf{c}}) = a \cdot \sum_{i=1}^{k_N} \left(\frac{n_{i+}}{N} \right)^2 + b \cdot \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{+j}}{N} \right)^2 - 2(a+b) \cdot \sum_{i=1}^{k_N} \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{ij}}{N} \right)^2.$$

VI 1

- La VI compara l'informazione in due partizioni con l'informazione condivisa fra esse:

$$VI(\mathbf{c}, \hat{\mathbf{c}}) = H(\mathbf{c}) + H(\hat{\mathbf{c}}) - 2I(\mathbf{c}, \hat{\mathbf{c}})$$

- Si dimostra che la partizione ottima che minimizza la perdita VI attesa a posteriori è la stessa che minimizza l'espressione:

$$\mathbf{c}^* = \operatorname{argmin}_{\hat{\mathbf{c}}} \sum_{n=1}^N \log \left(\sum_{n'=1}^N \mathbf{1}(\hat{c}_n = \hat{c}_{n'}) \right) - 2 \sum_{n=1}^N \mathbb{E} \left[\log \left(\sum_{n'=1}^N \mathbf{1}(c_{n'} = c_n, \hat{c}_{n'} = \hat{c}_n) \right) | \mathcal{D} \right].$$

- Problema: Valutare questo termine ha una complessità computazionale $O(MN^2)$

VI 2

- Soluzione: Applicare la disuguaglianza di Jensen per scambiare logaritmo e valore atteso calcolando un limite inferiore. Allora vale che:

$$\hat{\mathbf{c}}^* = \operatorname{argmin}_{\hat{\mathbf{c}}} \sum_{n=1}^N \log \left(\sum_{n'=1}^N \mathbf{1}(\hat{c}_n = \hat{c}'_{n'}) \right) - 2 \sum_{n=1}^N \log \left(\sum_{n'=1}^N \hat{\pi}_{nn'} \mathbf{1}(\hat{c}_{n'} = \hat{c}_n) \right).$$

- $\hat{\pi}_{nn'}$ elemento (nn') della *psm*, che può essere calcolata prima dell'ottimizzazione così riducendo la complessità computazionale a $O(N^2)$.

Lattice Theory

- Per ogni coppia di partizioni $\mathbf{c}, \hat{\mathbf{c}} \in \mathbf{C}$, $\mathbf{c}$ è detto coperto da $\hat{\mathbf{c}}$, scritto come $\mathbf{c} \prec \hat{\mathbf{c}}$, se $\mathbf{c} < \hat{\mathbf{c}}$ e non c'è nessun $\hat{\hat{\mathbf{c}}} \in \mathbf{C}$ tale che $\mathbf{c} < \hat{\hat{\mathbf{c}}} < \hat{\mathbf{c}}$.
- Per ogni coppia di partizioni $\mathbf{c}, \hat{\mathbf{c}} \in \mathbf{C}$, $\mathbf{c} \leq \hat{\mathbf{c}}$, se per ogni $i = 1, \dots, k_N$ se vale che $C_i \subseteq \hat{C}_j$ per qualche $j = 1, \dots, \hat{k}_N$.
- Il Diagramma di Hasse è definito secondo questa relazione d'ordine.
- Entrambe le metriche VI e Binder sono allineate con il reticolo delle partizioni.

Diagramma di Hasse

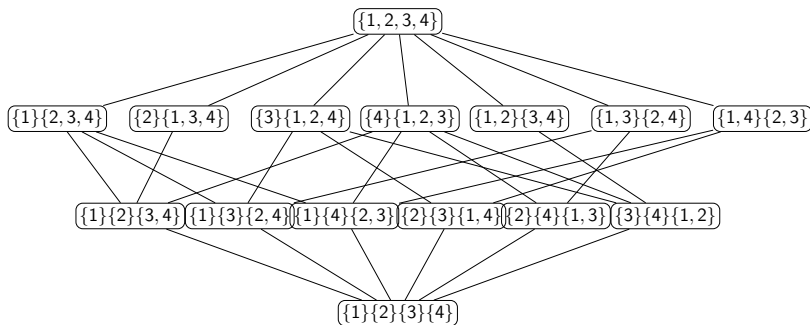


Figura: Diagramma di Hasse per un reticolo di partizioni per un campione di $N = 4$. La linea indica una relazione di copertura

Algoritmo Greedy Search

- Obiettivo: trovare $\mathbf{c}^* = \operatorname{argmin}_{\hat{\mathbf{c}} \in \mathbf{C}} \mathbb{E}[L(\mathbf{c}, \hat{\mathbf{c}}) | \mathcal{D}]$
- Come?
Considero tutte le possibili partizioni $\mathbf{c}$ dei dati e sfrutto le relazioni di copertura.

Algoritmo Greedy Search

- Dato $\hat{\mathbf{c}}$ considero:
 - 1 $\mathbf{c}_j$ l-esima partizione più vicina tale che: $\mathbf{c}_j \prec \hat{\mathbf{c}}$
 - 2 $\mathbf{c}_r$ l-esima partizione più vicina tale che $\hat{\mathbf{c}} \prec \mathbf{c}_r$
- Per ogni partizione visitata calcolo $\mathbb{E}[L(\mathbf{c}, \hat{\mathbf{c}})|\mathcal{D}]$
- Scelgo $\mathbf{c}^*$ che minimizza la perdita attesa a posteriori.

In sintesi

- L'algoritmo Greedy è una procedura che effettua piccoli aggiornamenti degli ottimi locali.
- Ad ogni step dell'algoritmo si cerca di minimizzare la perdita attesa.
- L'algoritmo finisce quando non vi sono riduzioni significative della perdita attesa a posteriori.

Vantaggi e svantaggi

- Vantaggi: possono essere considerate tutte le possibili partizioni e non soltanto quelle esplorate durante il campionamento MCMC.
- Svantaggi:
 - 1 forte dipendenza dalla partizione iniziale
 - 2 problemi di minimi locali.
- Soluzioni: inizializzazione randomica dell'algoritmo



Credible Balls

- Problema: come descrivere l'incertezza presente nella struttura dei cluster?

Heat map

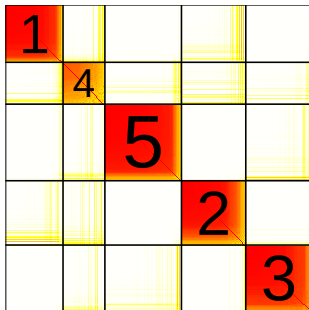


Figura: Heat map della poster similarity matrix

Credible Balls

Definisco le credible balls di livello $1 - \alpha$, con $\alpha \in [0, 1]$ piccolo come:

$$Ball_{\epsilon^*}(\mathbf{c}^*) = \{\mathbf{c} : d(\mathbf{c}^*, \mathbf{c}) \leq \epsilon^*\}$$

dove ϵ^* è il più piccolo $\epsilon \geq 0$ tale per cui $P(Ball_{\epsilon^*}(\mathbf{c}^*)|\mathcal{D}) \geq 1 - \alpha$.

Stima delle Credible Balls

- La stima delle credible balls avviene attraverso il campionamento MCMC.
- **Passaggi:**
 - 1 Calcolo la distanza fra tutti i campioni MCMC $\mathbf{c}$ e $\mathbf{c}^*$.
 - 2 Successivamente, per ogni $\epsilon > 0$, calcolo:

$$P(\text{Ball}_\epsilon(\mathbf{c}^*) \mid \mathcal{D}) = \mathbb{E}[\mathbf{1}(L(\mathbf{c}^*, \mathbf{c}) \leq \epsilon) \mid \mathcal{D}]$$
$$\approx \frac{1}{M} \sum_{m=1}^M \mathbf{1}(d(\mathbf{c}^*, \mathbf{c}^{(m)}) \leq \epsilon).$$

Mentre ϵ^* il più piccolo $\epsilon > 0$ tale per cui

$$\frac{1}{M} \sum_{m=1}^M \mathbf{1}(d(\mathbf{c}^*, \mathbf{c}^{(m)}) \leq \epsilon) \geq 1 - \alpha$$

Limiti verticali e orizzontali

I limiti verticali e orizzontali caratterizzano le crediball balls, in particolari sono 3:

- **Limite verticale inferiore:** partizione nella credible ball con il maggior numero di cluster che è più distante da $\mathbf{c}^*$;
- **Limite verticale superiore:** partizione nella credible ball con il minor numero di cluster che è più distante da $\mathbf{c}^*$;
- **Limite orizzontale:** partizione nella credible ball più distante da $\mathbf{c}^*$.



Fasi dell'Algoritmo Salso

L'algoritmo Salso è composto da 4 fasi principali:

- 1 Inizializzazione;
- 2 Ottimizzazione sweetening;
- 3 Aggiornamenti zealous;
- 4 Registrazione finale

Fase 1: Inizializzazione

Ad ogni iterazione dell'algoritmo vengono proposti due metodi di inizializzazione per l'allocazione delle etichette delle osservazioni nei cluster:

- 1 Allocazione sequenziale;
- 2 Campionamento uniforme.

Il metodo utilizzato viene scelto in maniera casuale dall'algoritmo con probabilità di scegliere l'allocazione sequenziale pari a p_{SA} .

Fase 2: ottimizzazione sweetening

- Ogni osservazione viene riallocata, una volta, ad ogni cluster;
- molto simile all'allocazione;
- l'obiettivo è la minimizzazione della perdita attesa a posteriori;
- procedimento termina quando non vi sono cambiamenti significativi nella stima Monte Carlo della perdita attesa.

Fase 3: Aggiornamenti zealous

- E' la fase computazionalmente più onerosa;
- obiettivo: superare i problemi di minimo locale;
- In modo casuale distruggo un cluster, rimuovo i suoi componenti e li rialloco con il metodo dell'allocazione sequenziale.
- Confronto la stima Monte Carlo della perdita attesa generata dall'aggiornamento zealous e quella precedente. Se non ci sono stati miglioramenti torno al punto di partenza.

Fase 4 e osservazioni

- Nella quarta fase dell'algoritmo semplicemente si registra il risultato ottenuto.
- Osservazioni: l'algoritmo è facilmente parallelizzabile, ogni iterazione è indipendente dalle altre.

Dati-I

Sono stati generati $N = 300$ osservazioni da:

$$X_i \stackrel{\text{iid}}{\sim} \sum_{j=1}^5 \frac{1}{5} \mathcal{N} \left(\begin{bmatrix} \mu_{j1} \\ \mu_{j2} \\ \mu_{j3} \end{bmatrix}, \begin{bmatrix} \sigma_j^2 & 0 & 0 \\ 0 & \sigma_j^2 & 0 \\ 0 & 0 & \sigma_j^2 \end{bmatrix} \right).$$

Dati-II

	j = 1	j = 2	j = 3	j = 4	j = 5
μ_1	-2	2	-2	2	0
μ_2	-2	-2	2	2	0
μ_3	-2	2	-2	2	0

Tabella: Valori del vettore delle medie per ciascuna componente

	j = 1	j = 2	j = 3	j = 4	j = 5
σ_j^2	1.00	1.25	1.00	1.25	0.50

Tabella: Valori delle varianze σ_j^2 per ciascuna componente

Dati-III

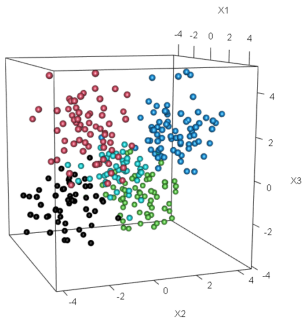
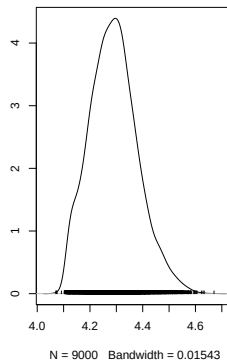
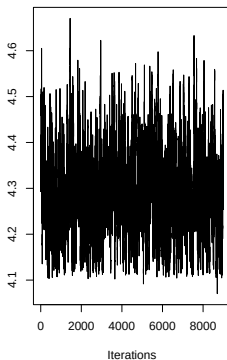


Figura: Scatterplot tridimensionale dei dati simulati per $N = 300$

Simulazioni MCMC-I

- I dati sono stati modellato attraverso un DPMM con kernel gaussiano.
- In particolare è stato utilizzato un Diagonal Location Scale model;
- Il parametro di concentrazione α del processo di Dirichlet è stato stabilito da una Hyperprior Gamma(1,1)
- La misura base è stata assunta come prodotto di normali-gamma inverse indipendenti.

Simulazioni MCMC-II



Simulazioni MCMC-III

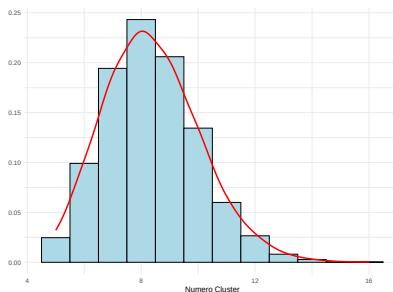


Figura: Stima della distribuzione a posteriori del numero di cluster

Greedy Search: VI

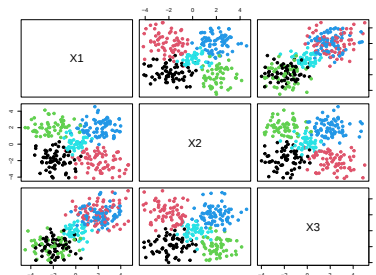
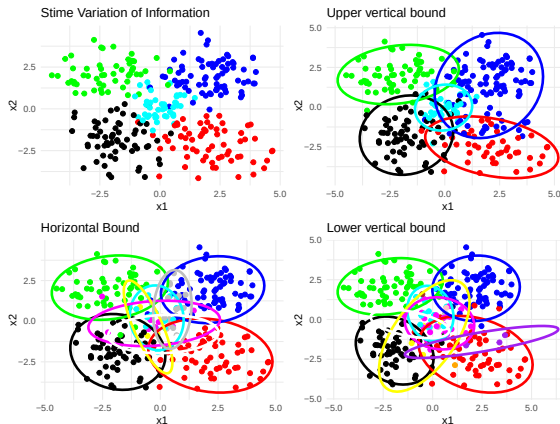


Figura: Proiezioni bidimensionali dei cluster ottenuti attraverso l'algoritmo Greedy Search e la funzione di perdita VI

Credible Balls: VI



Binder: Greedy Search e credible balls

- L'algoritmo Greedy Search applicato alla funzione di perdita Binder ha portato una partizione ottima composta da 24 cluster;
- In tabella sono rappresenti i limiti delle credible balls differenziate per le funzioni di perdita Binder e VI:

Loss	Upper		Lower		Horizontal	
	k_N^u	$d(c^*, c_u)$	k_N^l	$d(c^*, c_l)$	k_N^h	$d(c^*, c_h)$
Binder	5	1.23	16	1.11	12	1.28
VI	5	1.03	16	1.07	9	1.21

Tabella: Tabella riassuntiva dei risultati ottenuti attraverso le credible balls

Algoritmo Salso

- L'algoritmo salso è stato implementato considerando come parametri: $nRuns = 16$, $maxZeaoulous = 10$, $p_{SA} = 0.5$.
- Le funzioni di perdita utilizzate sono state: Binder simmetrica, Binder generalizzata ($a = 1.4666667$, $b = 1$) e VI;
- Il numero dei cluster ottenuti è stato rispettivamente: 16 per Binder simmetrica, 5 per Binder generalizzata e 5 per VI;
- Si osserva come scegliere $a > b$ porti ad una riduzione del numero di cluster stimata.

Confronto al variare di N-I

N	Algoritmo	k_N	N_i	$B(\hat{c}, c)$	$VI(\hat{c}, c)$	ARI	Sys Time
$N = 300$	Greedy VI	5	13	0.0326	0.5323	0.8985	25.0549
	Greedy Binder	24	27	0.0384	0.7658	0.8753	32.1067
	Salso VI	5	13	0.0326	0.5323	0.8985	4.2243
	Salso Binder ($a = 1$)	16	25	0.0377	0.7310	0.8787	132.0422
	Salso Binder ($a = 1.46$)	5	14	0.0352	0.5541	0.8909	20.0045
$N = 600$	Greedy VI	5	31	0.0391	0.6605	0.8777	117.7035
	Greedy Binder	24	279	0.0413	0.8061	0.8666	190.9277
	Salso VI	5	34	0.0428	0.7048	0.8660	16.1122
	Salso Binder ($a = 1$)	16	45	0.0419	0.7849	0.8654	341.1415
	Salso Binder ($a = 1.46$)	5	34	0.0424	0.7217	0.8672	59.5962

Confronto al variare di N-II

N	Algoritmo	k_N	N_i	$B(\hat{c}, c)$	$VI(\hat{c}, c)$	ARI	Sys TIme
$N = 900$	Greedy VI	5	48	0.0408	0.7207	0.8724	420.2393
	Greedy Binder	32	282	0.0422	0.8815	0.8639	455.5567
	Salso VI	5	52	0.0443	0.7638	0.8614	35.9570
	Salso Binder ($a = 1$)	17	67	0.0430	0.8666	0.8622	432.0280
	Salso Binder ($a = 1.46$)	5	51	0.0433	0.7641	0.8645	151.4777
$N = 1600$	Greedy VI	5	87	0.0420	0.7399	0.8688	3692.0178
	Greedy Binder	41	637	0.0408	0.8563	0.8690	2967.4418
	Salso VI	5	84	0.0408	0.7079	0.8725	66.1045
	Salso Binder ($a = 1$)	17	110	0.0414	0.8419	0.8678	1194.4606
	Salso Binder ($a = 1.46$)	5	86	0.0416	0.7328	0.8701	360.4050

Osservazioni finali

- I due algoritmi si differenziano per tempi computazionali e per precisione al crescere di N ;
- l'algoritmo Greedy con funzione di perdita VI è ottimo quando i cluster sono ben separati tra loro;
- aggiungendo rumore tra i dati i risultati ottenuti non coincidono con le attese;
- l'algoritmo Salso è più adatto a trattare strutture dati complesse.

I due dataset

- Country-Data, dataset messo a disposizione da HELP International. Contiene 9 variabili per 167 nazioni. Le osservazioni si riferiscono al 2010.
- Dati estrapolati dal sito Web della World Bank Group. Contiene 7 variabili per 159 nazioni. Le osservazioni sono aggiornate al 2022.
- Si è considerato un modello mistura DP Diagonal Location-Scale.
- Per fare inferenza ci si è basati sulla loss VI e sull'algoritmo Salso.

A world map illustrating the global distribution of the six major world religions. The map uses a color-coded system: Christianity is represented in red, Islam in yellow, Hinduism in orange, Buddhism in green, Judaism in purple, and Sikhism in blue. The distribution shows Christianity as the most widespread, covering large areas in Europe, North America, and parts of Africa and South America. Islam is concentrated in the Middle East, North Africa, and parts of Asia and Southeast Asia. Hinduism is primarily found in South Asia. Buddhism is concentrated in East Asia and Southeast Asia. Judaism and Sikhism are represented by small, localized areas in the Middle East and South Asia, respectively.



Figura: Partizione del Mondo nel 2010



Figura: Partizione del Mondo nel 2022

Casi particolari: Cina, Arabia Saudita e Irlanda

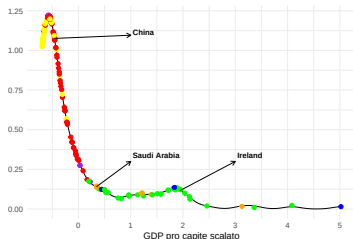


Figura: Densità GDP pro capite nel 2010

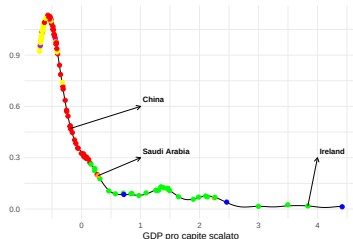


Figura: Densità GDP pro capite nel 2022

Limes, Geopolitica e Statistica bayesiana Non Parametrica

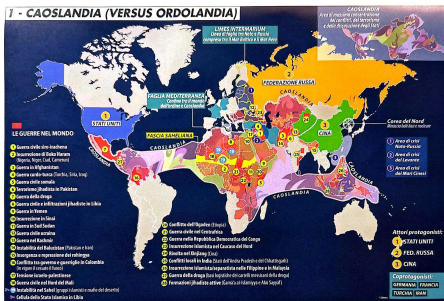


Figura: Limes: Caoslandia VS Ordolandia

- L'utilizzo di modelli mistura di Processo di Dirichlet per clustering ha portato a risultati sensati!